Buyer Lead Intake Platform

1. Overall Approach and Design Decisions

1.1 What Was Built

The platform is a two-agent LLM pipeline that converts raw, multi-channel buyer inquiries — emails, website forms, referrals — into structured, agent-ready property briefings. A Parser Agent (Gemini Flash, temperature 0.1) extracts a BuyerProfile from free-text, then a hybrid vector + hard-filter search retrieves the most relevant MLS listings, and a Strategist Agent (Gemini Flash, temperature 0.2) synthesises a full LeadBrief with property recommendations, strategic advice, a ready-to-paste follow-up message, and risk flags. The Streamlit front end exposes the full pipeline: lead pool sidebar, KPI metrics, buyer profile panel, strategic advice, property match cards with score breakdowns, and telemetry for debugging.

1.2 Key Design Decisions

Two-agent architecture instead of one: Separating extraction (Parser) from synthesis (Strategist) lets each agent run at the lowest necessary temperature — 0.1 for structured extraction, 0.2 for advice and follow-up copy — and allows MLS data to be injected between the two calls, which a single chain-of-thought approach would not support cleanly.

Hybrid vector + hard-filter search: A pure keyword store would miss semantic equivalences; a pure vector store would surface semantically rich listings that are over budget or under-bedroomed. The hybrid approach applies hard filters (budget gate at 105%/120%, bedroom gate with ±1 fallback) to enforce non-negotiables, then ranks by cosine similarity over Vertex AI text-embedding-004 vectors. Blended score weights: 50% vector similarity, 20% feature coverage, 20% budget efficiency, 10% bedroom fit.

Pydantic schemas with field validators: LLMs can return urgency scores of 11, negative budgets, or placeholder names. Every field in BuyerProfile and LeadBrief has a validator that silently corrects or rejects nonsensical values, so downstream logic can treat schema outputs as trusted data.

Prompt injection sanitisation: Lead 6 (Aaron Cooper) included a classic injection attempt. A compiled regex strips known injection patterns from inquiry text before any LLM call, providing a fast, deterministic defence without an extra model call.

Three-layer hallucination fix for buyer names: Without intervention, the LLM hallucinated names in 10 of 12 leads. The fix uses three complementary layers: (1) buyer_name declared as Optional[str] in both schemas so the LLM outputs null when no name is present; (2) an explicit Strategist prompt instruction not to invent a name; and (3) the orchestrator unconditionally overwrites buyer_name with the authoritative value from the metadata dict after pipeline completion. All three together make the correct name deterministic.

reference_price field: Lead 5 (Robert Klein) mentioned a $1.25M listing while offering $950K. A dedicated reference_price field captures both signals, steering the embedding model toward the correct market tier rather than anchoring purely on the lower offer price.

Fixing 0-bedroom listings at load time: The CSV contained 4 listings with bedrooms=0 — bad data that passed every bedroom filter and surfaced as top vector matches. These are discarded at load time so they never enter the vector index. A belt-and-suspenders bedroom_score function also returns 0.0 for any listing with fewer than one bedroom in case future bad rows slip through.

Active-only inventory filter: The MLS CSV contains 299 total listings across three statuses: 208 Active, 53 Active Under Contract, and 38 Pending. Only listings with status Active are indexed and searchable. Active Under

Contract and Pending listings are excluded at load time because they are not available to new buyers — surfacing them as recommendations would mislead the agent and the buyer. This filter runs before vector indexing, so the embedding matrix and all downstream scoring operate exclusively on the 208 purchasable listings.

1.3 Tradeoffs

Prompt injection sanitisation — latency vs. token count: Before the regex sanitiser was introduced, Aaron Cooper's injection-laced message was passed in full to the Parser Agent. The LLM processed the injected instruction text, which drove parser latency to 9.72 seconds (vs. a ∼1.9 second average for clean leads) because the model was internally reasoning about the injected command before producing output. Token consumption was also elevated because the full unsanitised text, including the injected command, occupied input context. After the regex sanitiser was added, the injected fragment is stripped and replaced with a short [REDACTED] token before the text enters any prompt. The result is the opposite profile: parser latency for Lead 6 dropped back to the normal ∼10 second range (the slight increase over average is the regex pass itself, which is sub-millisecond), and input token count fell because the injected clause no longer fills prompt context. The tradeoff accepted here is coverage: the regex catches known, literalistic injection patterns but a sophisticated paraphrase or multilingual attempt could evade it. A production deployment would add a dedicated guard model call; the regex was chosen for speed and zero added latency during development.

Strategist listing cap — token reduction: The Strategist Agent prompt originally received all candidate listings returned by the search layer, with no cap. For low-specificity leads (no neighbourhood, no budget), the hard-filter fallback could return the full active inventory, sending up to 208 property JSON objects into the prompt. This inflated input token counts by roughly 3,000–5,000 tokens per such lead and degraded output quality because the LLM had to attend to a very large context before producing rationale. A cap of 8 listings (MAX_RESULTS=8) is now applied before serialisation in the orchestrator. This reduced strategist input tokens significantly for vague leads while producing no quality loss for well-specified ones — a good match never needs more than 8 candidates to make a confident recommendation.

Startup embedding cost vs. per-query latency: All 208 active listings are embedded via Vertex AI text-embedding-004 in a single batched API call at startup (∼1–3 seconds, ∼1 MB of float32 vectors in RAM). Every subsequent search requires only one additional embedding call — to embed the query text — because similarity is computed against the pre-built in-memory index using NumPy matrix multiplication. The alternative, embedding listings on demand per query, would add ∼1 second per search and multiply API calls with every lead processed. The chosen approach front-loads the cost once and amortises it across all queries in a session. At the current inventory size the in-memory NumPy index is faster and simpler than FAISS; FAISS or a managed vector database would be the correct choice at 100K+ listings.

Two-agent call overhead vs. output quality: The two-agent architecture doubles the minimum number of LLM calls per lead (one Parser call, one Strategist call). Each call carries its own latency and token cost. This was accepted because the quality improvement from separating extraction and synthesis is significant: the Parser can run at temperature 0.1 and focus entirely on structured JSON extraction, while the Strategist can write natural prose at temperature 0.2 without simultaneously tracking schema constraints. A single-call design would require a higher temperature compromise and produce either noisier structured data or blander prose.

The regex injection sanitiser trades pattern coverage for speed and zero added LLM cost; a production system would add a dedicated guard model call for high-risk channels.

2. Walkthrough of the 12 Lead Briefs

Each brief below summarises the key inputs, agent decisions, and any notable pipeline behaviour.

Lead 1: Marcus Thompson (LEAD-2026-001)

Relocating to Miami for a tech job. Urgency 8/10 (firm August deadline). Parser correctly extracted both neighbourhoods (Brickell, Downtown Miami) and all three amenity must-haves. Top match: 7267 Biscayne Blvd #725 at $648K with gym, bay view, and concierge — comfortably under budget. Clean, structured inquiry with no pipeline issues.

Lead 2: Patricia and David Chen (LEAD-2026-002)

Family of four relocating from Boston, 4+ bedrooms, pool non-negotiable, budget $2.3M, flexible timeline. Urgency 3/10. Top match: 2470 Ponce de Leon Blvd at $1.458M in Coral Gables with a pool. Strategist correctly recommended building rapport rather than pushing for an immediate showing, and flagged school-district verification given two elementary-age children.

Lead 3: Anonymous (LEAD-2026-003)

Anonymous form submission. 4-bedroom pool property with ocean view in Downtown Miami/Brickell — budget $250K, needed in one week. Urgency 10/10. The pipeline correctly flagged the severe budget-to-market mismatch (~8× below realistic pricing), an impossible one-week purchase timeline, and missing contact information. Properties shown were Tier 3 Broad Fallback. Strategist correctly advised a discovery call to reset expectations rather than sending listings.

Lead 4: Sofia Reyes (LEAD-2026-004)

One-sentence referral: interest in an investment property in Miami. No budget, no bedroom count, no neighbourhood. Urgency 3/10. Three risk flags: missing budget, missing investment criteria, no timeline. Listings surfaced were Tier 1 broad results. Strategist correctly recommended a qualification call before sending properties.

Lead 5: Robert Klein (LEAD-2026-005)

Mentions a listing at $1.25M and is considering offering $950K, seeking negotiation guidance. The reference_price field resolved the dual price signal, surfacing the target property itself plus comparable Mid-Beach listings. Risk flags: significant low-ball offer and professional disclosure concerns around seller motivation requests. Strategist advised explaining market dynamics rather than validating the low-ball strategy.

Lead 6: Aaron Cooper (LEAD-2026-006)

3-bedroom single-family home in Aventura/North Miami, $850K, garage required. Message contained a prompt injection attempt. The injection was detected and stripped before reaching the LLM; parser latency dropped from 9.72s (pre-fix) back to the normal range post-fix, and input tokens were reduced by removing the injected clause from the prompt context. The legitimate inquiry was processed normally. Top match: 4883 Hibiscus Drive at $485K with a garage.

Lead 7: Elena Vasquez (LEAD-2026-007)

Buying on behalf of elderly parents. 2BR, single-story or elevator access, near pharmacy/grocery/medical, under $600K. Urgency 5/10. Accessibility requirements were correctly extracted despite being non-standard real estate terms. Risk flags: MLS data does not consistently capture elevator/walkability specifics; and the apparent contradiction between ‘single-story’ and ‘elevator access’ was correctly surfaced for buyer clarification.

Lead 8: Jennifer Walsh (LEAD-2026-008)

Long conversational message. Family moving from Chicago with two kids, a dog, needing a pool, home office, and good schools in Coconut Grove/Coral Gables. Budget 'about $1.4M, ideally $1.2M.' Parser correctly set budget_max=1,400,000. Top match at $1.458M triggered a risk flag for exceeding budget; strategist recommended a budget flexibility conversation rather than presenting the overage as a non-issue.

Lead 9: Luis Hernandez (LEAD-2026-009)

Cash buyer, townhouse in Brickell, $750K max, 2–3 bedrooms, 2 parking spots. Urgency 5/10. Risk flag: $750K is significantly below current Brickell townhouse market entry (~$1M+). Pipeline surfaced Tier 3 Broad Fallback matches. Strategist correctly recommended educating Luis on market pricing and exploring alternative neighbourhoods or property types.

Lead 10: Karen O'Brien (LEAD-2026-010)

High-net-worth buyer from NYC. Luxury waterfront, Key Biscayne or Bal Harbour, 5+ bedrooms, private boat dock, budget up to $8M flexible, year-end closing. Urgency 8/10. Top match: 9231 West Avenue at $6.707M with private dock in Bal Harbour. Strategist correctly prioritised this as the highest-value lead, recommending an immediate call and virtual tour.

Lead 11: Priya Sharma (LEAD-2026-011)

First-time buyer, nervous tone. 1–2 bedroom condo under $400K, Wynwood preferred, pet-friendly (cat). Urgency 3/10. Risk flag: $400K is below current Wynwood condo pricing (~$500K+). Pipeline returned no viable matches — the correct outcome. Strategist recommended a gentle market reality conversation and exploring adjacent, more affordable neighbourhoods.

Lead 12: Michael Reeves (LEAD-2026-012)

Investor seeking cash-flowing rentals in Miami-Dade, multi-family or condos, $500K–$900K per property, planning 2–3 acquisitions over 6 months. Urgency 4/10. Risk flags: MLS data is primarily residential; clarification needed on whether strict multi-family or condos as rental vehicles are acceptable. Strategist correctly recommended leading with rental yield data rather than lifestyle features.

3. How AI Coding Tools Were Used

3.1 Which Tools

The primary AI coding assistant was Claude (Anthropic) via claude.ai and the Anthropic API. GitHub Copilot was used for IDE autocomplete on boilerplate-heavy sections such as Pydantic model definitions and Streamlit widget calls.

3.2 Where They Helped

AI assistance was most valuable in three areas: (1) generating initial Pydantic schema scaffolding for BuyerProfile, PropertyMatch, LeadBrief, and PerformanceMetrics; (2) writing and iterating the hybrid scoring logic in ingestion_vector.py, including articulating tradeoffs between different blend weights; and (3) drafting prompt templates and identifying which fields needed explicit null instructions — buyer_name being the key example.

3.3 Where Overrides Were Necessary

The AI tools consistently underestimated hallucination risk for buyer names — early drafts omitted the field entirely on the reasoning that the name would be 'in the metadata anyway.' Evaluation results proved this wrong, and the three-layer fix was developed by reasoning through the failure mode rather than accepting the AI's simpler

initial design. The 0-bedroom listing bug was caught through manual data inspection, not AI suggestion. The reference_price field originated from analysing Lead 5 specifically — the AI's default design would have anchored the search on the wrong price tier, a mistake that required domain reasoning about buyer communication patterns to identify and correct.

4. What Would Be Done Differently or Built Next

4.1 What Would Be Done Differently

Separate the vector index build from the MLS data load so the expensive embedding step is skipped on restarts where the CSV has not changed — cache the raw DataFrame and embedding matrix separately, keyed on a hash of the CSV content. Replace the prompt injection regex with a dedicated guard model or classifier that catches paraphrased and multilingual injection attempts. Move to parallel lead processing with a shared rate-limit token bucket for scale rather than the current sequential adaptive back-off.

4.2 What Would Be Built Next

Four priorities in order: (1) A feedback loop from the agent back into the system — capturing which properties were shown, led to showings, and led to offers — to enable blend-weight fine-tuning based on actual conversion rather than theoretical relevance. (2) Live MLS API integration with incremental vector index updates, replacing the static CSV. (3) A dedicated follow-up message generation call with access to prior conversation history for genuinely personalised output. (4) Structured severity levels (low/medium/high/critical) on risk flags, enabling automatic routing of high-risk leads to senior agents.